

ARIA

Adaptive Research Intelligence Architecture

Sr. AI Engineer Technical Assessment

May 2026

Table of Contents

Section 1: Problem Statement	3
Section 2: The Core Challenge	3
Section 3: Required Components	3
3.1 Orchestrator Agent	3
3.2 Specialist Agents (minimum 3 types)	3
3.3 Memory & State Layer	4
3.4 Quality Evaluator	4
3.5 Final Report	4
Section 4: Hard Constraints	5
Section 5: What to Submit	5
5.1 Working Codebase	5
5.2 Design Document (4–6 pages)	5
Section 6: Evaluation Rubric	5
Technical Depth (40%)	5
Systems Thinking (25%)	6
Research Awareness (20%)	6
Engineering Quality (15%)	6
Section 7: Bonus Challenges	7
Section 8: Reference Papers	7
Section 9: What This Problem Reveals	7

Section 1: Problem Statement

You are tasked with building **ARIA**, a multi-agent system that performs automated deep research on a given query by coordinating specialized agents, synthesizing conflicting information, and producing a structured, citation-backed research report — while continuously self-evaluating and replanning when results are poor.

This problem sits at the intersection of three active research areas:

- Retrieval-Augmented Generation (RAG)
- Multi-agent coordination
- LLM self-reflection / critique

Section 2: The Core Challenge

Given a complex, open-ended research query such as:

“What are the current limitations of transformer-based long-context reasoning, and what architectural directions show the most promise for solving them?”

ARIA must:

1. Decompose the query into a dependency graph of sub-questions
2. Dispatch specialized agents to answer each sub-question in parallel
3. Detect and resolve conflicts when agents return contradictory findings
4. Synthesize a final report with inline citations and confidence scores
5. Self-critique the report against a rubric and decide whether to re-run failing sections
6. Terminate gracefully with a quality gate — not just when “done”, but when the output meets a minimum quality threshold

Section 3: Required Components

3.1 Orchestrator Agent

- Parses the top-level query into a DAG of sub-tasks
- Assigns tasks to specialist agents with explicit context budgets
- Handles dynamic replanning: if an agent returns low-confidence results, the orchestrator must decide to retry, rephrase, or merge with another agent’s output
- Implements a termination policy (not infinite loops)

3.2 Specialist Agents (minimum 3 types)

- **Retrieval Agent:** Fetches and chunks relevant content from web search or a provided document corpus. Must implement a retrieval strategy beyond naive top-k (e.g., HyDE, iterative retrieval, reranking)

- **Critique Agent:** Given two conflicting claims from different sources, reasons about which is more credible and why — producing a structured resolution with evidence
- **Synthesis Agent:** Merges outputs from multiple retrieval agents into a coherent section, deduplicates, and assigns citations

3.3 Memory & State Layer

- Shared working memory accessible to all agents (what has been retrieved, what conflicts were resolved, what sections are complete)
- Agents must not re-retrieve already-known information (redundancy detection)
- Memory must be serializable — if the run is interrupted and resumed, state must persist

3.4 Quality Evaluator

- A lightweight LLM-as-judge layer that scores each section on: factual grounding, coverage, coherence, and citation quality
- Scores feed back into the orchestrator's replanning decisions
- The system must expose these scores in the final output

3.5 Final Report

- Structured Markdown with: executive summary, per-section findings, conflict resolutions, confidence scores, and a bibliography
- Must be reproducible: running twice on the same query should produce structurally equivalent outputs (not necessarily identical tokens)

Section 4: Hard Constraints

Constraint	Requirement
No monolithic LLM call	Each major step must be a distinct agent with its own prompt, context, and output schema
Tool use required	Agents must use at least 2 external tools (web search, file I/O, vector DB, etc.)
No infinite retry loops	Orchestrator must have a hard budget (max 3 replan cycles per sub-task)
Structured outputs	All inter-agent communication must use typed schemas (Pydantic, TypedDict, or equivalent)
Evaluation is first-class	Quality scores must be computed and logged — not optional

Section 5: What to Submit

5.1 Working Codebase

- Runnable with a single command against at least one test query
- Clear README with setup, architecture diagram, and how to run
- Reasonable test coverage on the orchestrator’s replanning logic and conflict resolution

5.2 Design Document (4–6 pages)

Must address:

- **Agent topology:** Why this structure? What alternatives did you consider and reject?
- **Conflict resolution strategy:** How does the Critique Agent decide credibility? What are the failure modes?
- **Memory design:** What consistency guarantees does shared memory provide? What breaks under concurrency?
- **Termination & quality:** How do you prevent the system from running forever or stopping too early?
- **What you’d do with 2 more weeks:** Specific, technically grounded — not “scale it up”

Section 6: Evaluation Rubric

Technical Depth (40%)

- Is the orchestrator’s replanning logic non-trivial? Does it reason about why an agent failed?
- Is retrieval beyond naive RAG? (HyDE, iterative, reranking, query decomposition)
- Is conflict resolution principled, or just “take the most recent”?

Systems Thinking (25%)

- Does the memory layer handle concurrency correctly?
- Are agent boundaries clean — or does the orchestrator do work agents should?
- Does the termination policy actually work under adversarial inputs?

Research Awareness (20%)

- Does the design doc reference and engage with relevant literature (e.g., ReAct, STORM, Self-RAG, LLM-as-Judge, Constitutional AI)?
- Are tradeoffs discussed honestly — not just “our approach is best”?

Engineering Quality (15%)

- Code is readable and structured for a team, not a hackathon
- Typed interfaces between agents
- At least one non-trivial test

Section 7: Bonus Challenges

- **Adaptive context budgeting:** Orchestrator dynamically allocates token budgets to agents based on sub-task complexity
- **Citation verification:** A separate agent spot-checks whether retrieved claims actually appear in the cited source
- **Multi-hop reasoning:** The system handles queries where answering sub-question B requires the answer to sub-question A first (true DAG execution, not parallel-only)
- **Streaming output:** Report sections stream to the user as they're completed, not at the end

Section 8: Reference Papers

The following papers are provided to the candidate:

- **STORM** (Stanford, 2024) — Wikipedia-scale article generation via multi-agent outline + retrieval
- **Self-RAG** (Asai et al., 2023) — Retrieval with self-reflection tokens
- **LLM-as-Judge** (Zheng et al., 2023) — Using LLMs to evaluate LLM outputs
- **ReAct** (Yao et al., 2022) — Reasoning + acting in language models

Section 9: What This Problem Reveals

Trait	How it shows up
Can they architect, not just code?	Agent topology and memory design
Do they know the research?	Design doc references + retrieval strategy choice
Do they handle failure modes?	Replanning logic, termination policy, conflict resolution
Do they think about eval?	Quality evaluator — is it bolted on or integral?
Sr. vs Mid-level gap	Mid-level builds the happy path. Sr. engineers design for when it breaks.